Taking inspiration from the Heroku "B-trees and Sorting" post from earlier, create an index on trips.completed_at that will be used by the ORDER BY clause from the query above. This Index orders the NULL completed_at records at the end. Run this statement in psql to create the index:

```
CREATE INDEX trips_completed_at_index
ON trips(completed_at DESC NULLS LAST);

ANALYZE trips;
```

Run the query again. The query is now much faster and the execution plan lists the index "trips_completed_at_index". An Index Scan was performed for the Sort Operation with the Sort Key: completed_at.

You've now seen a less common way to use an Index. Indexes usage might be more common, including to cover multiple columns as a Covering Index. How does that work? Read on to learn more.

Using Covering Indexes

When the fields being requested match exactly with the fields covered by an index, this type of Index is called a "Covering Index."

From PostgreSQL version 11 the INCLUDE[16] keyword can be used for Covering Indexes to enable Index Only Scans. The INCLUDE keyword is a little different from a Multicolumn index but they are two ways to create a Covering Index.

With the INCLUDE keyword, columns can be added to a special payload area. Payload columns are not filtered on in a WHERE clause but are available if chosen in the SELECT part of a query.

In Active Record it's less common to explicitly select columns. Consider places in your codebase that don't select specific columns. Look at the minimum set of columns needed, and what columns are already Indexed. You may want to add missing Covering Indexes for performance sensitive areas that select a few columns.

When columns match a Covering Index, PostgreSQL can perform the very efficient Index Only Scan type and get all of the data needed from the Index.

It's time to create examples of these scenarios to better understand them.

Create a Multicolumn index on first_name and last_name and then analyze the table, by running these statements from psql:

```
CREATE INDEX users_fname_lname_multi_idx ON users (first_name, last_name);
```

16. https://www.postgresql.org/docs/11/indexes-index-only-scans.html